

SEMINARARBEIT

Im Studiengang FH Technikum Wien, Bachelorstudiengang BIF
Lehrveranstaltung BIF-VZ-5-WS2025-INFC-EN

Terraform Workshop Protocol

Ausgeführt von: David Veigel

Kevin Forter

Personenkennzeichen: XXXXXXXXXXXXXXXX

BegutachterIn:

Wien, den 28. November 2025

Inhaltsverzeichnis

1 Workshop 1: Terraform Basics

In this workshop, we set up a simple web server on AWS using Terraform. The goal is to provision an EC2 instance in the default VPC, install an Apache web server using user data, and verify that it is reachable.

1.1 Configuration

1.1.1 Provider and Region

We configure the AWS provider to use the `us-east-1` region, as specified in the requirements.

```
provider "aws" {  
    region = "us-east-1"  
}
```

1.1.2 VPC and Subnets

Instead of creating a new network, we use the existing default VPC and its subnets. This simplifies the setup for this initial workshop.

```
data "aws_vpc" "default" {  
    default = true  
}  
  
data "aws_subnets" "default_vpc_subnets" {  
    filter {  
        name      = "vpc-id"  
        values    = [data.aws_vpc.default.id]  
    }  
}
```

1.1.3 Security Group

To make our web server accessible, we create a security group. We allow inbound TCP traffic on port 80 (HTTP) from any IP address (`0.0.0.0/0`). We also allow all outbound traffic to ensure the server can download updates and packages.

```

resource "aws_security_group" "web_sg" {
  name          = "tf-web-sg"
  description   = "Allow_HTTP_in,_all_egress"
  vpc_id        = data.aws_vpc.default.id

  ingress {
    description      = "HTTP"
    from_port        = 80
    to_port          = 80
    protocol          = "tcp"
    cidr_blocks       = ["0.0.0.0/0"]
  }

  egress {
    description      = "All_egress"
    from_port        = 0
    to_port          = 0
    protocol          = "-1"
    cidr_blocks       = ["0.0.0.0/0"]
  }
}

```

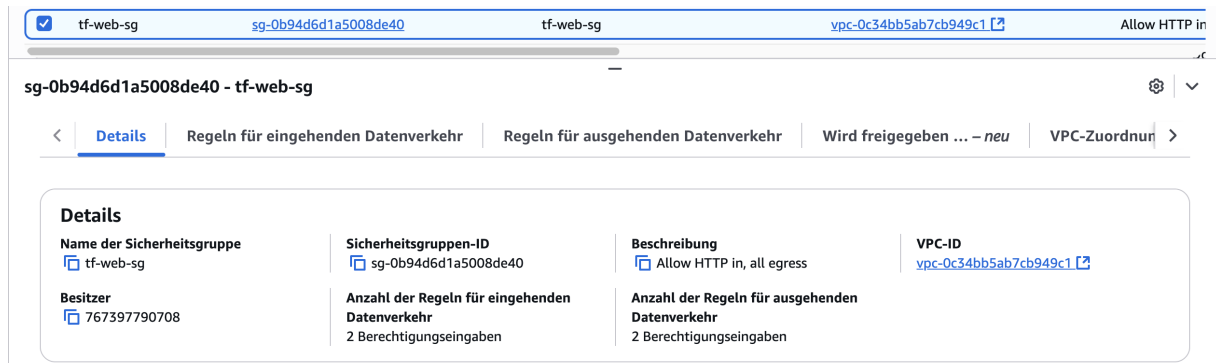


Abbildung 1: Security Group Configuration in AWS Console

1.1.4 EC2 Instance

We provision an EC2 instance using the `t2.micro` instance type, which is eligible for the free tier. We dynamically fetch the latest Ubuntu 22.04 LTS AMI. The `user_data` script is used to automatically install and start the Apache web server upon launch.

```

resource "aws_instance" "web" {
  ami          = data.aws_ami.ubuntu.id
  instance_type = "t2.micro"
  subnet_id    = data.aws_subnets.default_vpc_subnets.ids[0]
}

```

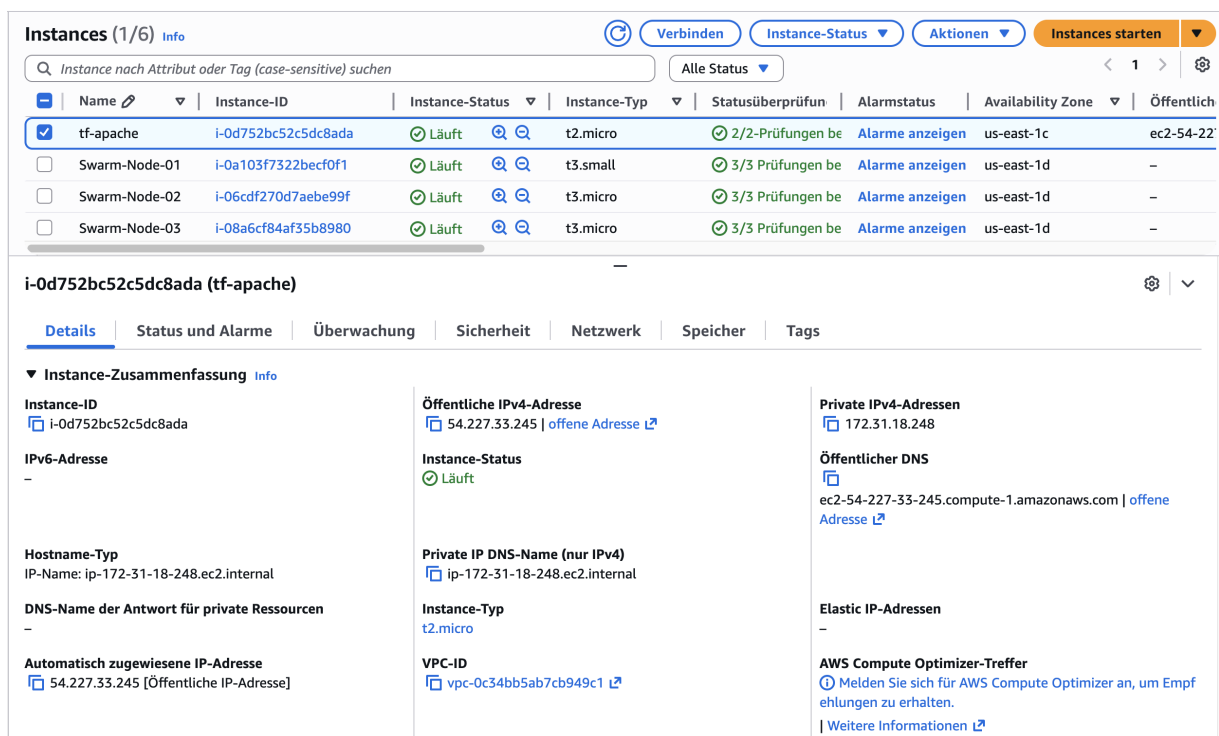
```

vpc_security_group_ids      = [aws_security_group.web_sg.id]
associate_public_ip_address = true

user_data = <<-EOF
#!/bin/bash
sudo apt-get update
sudo apt-get install -y apache2
sudo systemctl start apache2
sudo systemctl enable apache2
echo "<h1>Hello_World</h1>" | sudo tee /var/www/html/index.html
EOF

tags = {
  Name = "tf-apache"
}

```



Instances (1/6) Info

Q Instance nach Attribut oder Tag (case-sensitive) suchen

Alle Status

Name	Instance-ID	Instance-Status	Instance-Typ	Statusüberprüfun	Alarmstatus	Availability Zone	Öffentlich
☒ tf-apache	i-0d752bc52c5dc8ada	Lauft	t2.micro	2/2-Prüfungen be	Alarme anzeigen	us-east-1c	ec2-54-227
☐ Swarm-Node-01	i-0a103f7322becf0f1	Lauft	t3.small	3/3 Prüfungen be	Alarme anzeigen	us-east-1d	-
☐ Swarm-Node-02	i-06cdf270d7aeb99f	Lauft	t3.micro	3/3 Prüfungen be	Alarme anzeigen	us-east-1d	-
☐ Swarm-Node-03	i-08a6cf84af35b8980	Lauft	t3.micro	3/3 Prüfungen be	Alarme anzeigen	us-east-1d	-

i-0d752bc52c5dc8ada (tf-apache)

Details | Status und Alarme | Überwachung | Sicherheit | Netzwerk | Speicher | Tags

▼ Instance-Zusammenfassung Info

Instance-ID i-0d752bc52c5dc8ada	Öffentliche IPv4-Adresse 54.227.33.245 offene Adresse	Private IPv4-Adressen 172.31.18.248
IPv6-Adresse -	Instance-Status Lauft	Öffentlicher DNS ec2-54-227-33-245.compute-1.amazonaws.com offene Adresse
Hostname-Typ IP-Name: ip-172-31-18-248.ec2.internal	Private IP DNS-Name (nur IPv4) ip-172-31-18-248.ec2.internal	Elastic IP-Adressen -
DNS-Name der Antwort für private Ressourcen -	Instance-Typ t2.micro	AWS Compute Optimizer-Treffer Melden Sie sich für AWS Compute Optimizer an, um Empfehlungen zu erhalten. Weitere Informationen
Automatisch zugewiesene IP-Adresse 54.227.33.245 [Öffentliche IP-Adresse]	VPC-ID vpc-0c34bb5ab7cb949c1	

Abbildung 2: EC2 Instance Running in AWS Console

1.1.5 Output and Verification

Finally, we output the public DNS of the instance to easily access it. We also include a `null_resource` to verify that the web server is actually reachable via HTTP.

```

output "instance_public_dns" {

```

```
description = "Public_DNS_of_the_EC2_instance"
value       = aws_instance.web.public_dns
}
```

1.2 Execution

We initialize the Terraform project to download the necessary providers.

```
> terraform init
Initializing the backend...
Initializing provider plugins...
- Finding hashicorp/aws versions matching ">= 5.0.0"...
- Finding latest version of hashicorp/null...
- Installing hashicorp/aws v6.20.0...
- Installed hashicorp/aws v6.20.0 (signed by HashiCorp)
- Installing hashicorp/null v3.2.4...
- Installed hashicorp/null v3.2.4 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

Abbildung 3: Terraform Init

Then we apply the configuration to create the resources.

```

> terraform apply -auto-approve
data.aws_vpc.default: Reading...
data.aws_ami.ubuntu: Reading...
data.aws_ami.ubuntu: Read complete after 4s [id=ami-0c398cb65a93047f2]
data.aws_vpc.default: Read complete after 7s [id=vpc-0c34bb5ab7cb949c1]
data.aws_subnets.default_vpc_subnets: Reading...
data.aws_subnets.default_vpc_subnets: Read complete after 1s [id=us-east-1]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# aws_instance.web will be created
+ resource "aws_instance" "web" {
  + ami                  = "ami-0c398cb65a93047f2"
  + arn                  = (known after apply)
  + associate_public_ip_address = true
  + availability_zone     = (known after apply)
  + disable_api_stop     = (known after apply)
  + disable_api_termination = (known after apply)
  + ebs_optimized        = (known after apply)
  + enable_primary_ipv6   = (known after apply)
  + force_destroy        = false
  + get_password_data     = false
  + host_id              = (known after apply)
  + host_resource_group_arn = (known after apply)
  + iam_instance_profile  = (known after apply)
  + id                   = (known after apply)
  + instance_initiated_shutdown_behavior = (known after apply)
  + instance_lifecycle   = (known after apply)
  + instance_state       = (known after apply)
  + instance_type        = "t2.micro"
  + ipv6_address_count   = (known after apply)
  + ipv6_addresses       = (known after apply)
  + key_name             = (known after apply)
  + monitoring           = (known after apply)
  + outpost_arn          = (known after apply)
  + password_data        = (known after apply)
  + placement_group      = (known after apply)
  + placement_group_id   = (known after apply)
  + placement_partition_number = (known after apply)
  + primary_network_interface_id = (known after apply)
  + private_dns          = (known after apply)
  + private_ip           = (known after apply)
  + public_dns           = (known after apply)
  + public_ip            = (known after apply)
  + region              = "us-east-1"
  + secondary_private_ips = (known after apply)
  + security_groups       = (known after apply)
  + source_dest_check     = true

```

Abbildung 4: Terraform Apply

2 Workshop 2: Advanced Networking

In the second workshop, we move away from the default VPC and build a custom network infrastructure. This includes creating a VPC, subnets, routing tables, and an Internet Gateway. We also assign a persistent Elastic IP to our web server.

2.1 Network Infrastructure

2.1.1 VPC and Internet Gateway

We create a new VPC with the CIDR block `10.0.0.0/16`. To enable communication with the internet, we attach an Internet Gateway to this VPC.

```
resource "aws_vpc" "web_vpc" {
  cidr_block      = "10.0.0.0/16"
  enable_dns_support = true
  enable_dns_hostnames = true
  tags = { Name = "tf-web-vpc" }
}

resource "aws_internet_gateway" "igw" {
  vpc_id = aws_vpc.web_vpc.id
  tags = { Name = "tf-web-igw" }
}
```

vpc-01642c1407e32f458 / tf-web-vpc Aktionen ▾

Details Info	
VPC-ID vpc-01642c1407e32f458	Status Available
DNS-Auflösung Aktiviert	Tenancy default
Haupt-Netzwerk-ACL acl-0f155e51d51550947	Standard-VPC Nein
IPv6 CIDR (Network Border Group) -	Nutzungsmetriken für Netzwerkadressen Deaktiviert
Verschlüsselungssteuerungs-ID -	Verschlüsselungssteuerungsmodus -
Öffentlichen Zugriff blockieren Aus DHCP-Optionssatz dopt-0804897584b01e432 IPv4-CIDR 10.0.0.0/16 Regelgruppen für Route-53-Resolver-DNS-Firewall Fehler beim Laden der Regelgruppen	
DNS-Hostnamen Aktiviert Haupt-Routing-Tabelle rtb-07e4fa61f19867c7f IPv6-Pool - Besitzer-ID 767397790708	

Ressourcenzuordnung Info | CIDRs | Flow-Protokolle | Tags | Integrationen

Ressourcenzuordnung Info Alle Details anzeigen

VPC
Ihr virtuelles AWS-Netzwerk
tf-web-vpc

Subnetze (1)
Subnetze innerhalb dieser VPC
us-east-1a
tf-web-subnet

Routing-Tabellen (2)
Netzwerkdatenverkehr an Ressourcen weiterleiten
tf-web-rt
rtb-07e4fa61f19867c7f

Abbildung 5: Custom VPC Created

igw-0c40f497ee5149ed6 / tf-web-igw Aktionen ▾

Details Info	
Internet-Gateway-ID igw-0c40f497ee5149ed6	Status Attached
VPC-ID vpc-01642c1407e32f458 tf-web-vpc	Eigentümer 767397790708

Tags (1) Tags verwalten

Tags suchen

Schlüssel	Wert
Name	tf-web-igw

Abbildung 6: Internet Gateway Attached

2.1.2 Subnet and Routing

We create a subnet (10.0.1.0/24) within our VPC. To allow traffic from the subnet to reach the internet, we create a custom route table that routes all traffic (0.0.0.0/0) to the Internet Gateway, and associate it with our subnet.

```
resource "aws_route_table" "web_rt" {
  vpc_id = aws_vpc.web_vpc.id
  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.igw.id
  }
  tags = { Name = "tf-web-rt" }
}

resource "aws_subnet" "web_subnet" {
  vpc_id            = aws_vpc.web_vpc.id
  cidr_block        = "10.0.1.0/24"
  availability_zone  = "us-east-1a"
  map_public_ip_on_launch = false
  tags = { Name = "tf-web-subnet" }
}

resource "aws_route_table_association" "web_rta" {
  subnet_id      = aws_subnet.web_subnet.id
  route_table_id = aws_route_table.web_rt.id
}
```

subnet-003febe3900aa2e0f / tf-web-subnet Aktionen

Details

Subnetz-ID
subnet-003febe3900aa2e0f

IPv4-CIDR
10.0.1.0/24

Verfügbare Zone
us-east-1 (us-east-1a)

Netzwerk-ACL
acl-0f155e51d51550947

Kundeneigene IPv4-Adresse automatisch zuweisen
Nein

IPv6 CIDR-Reservierungen
-

Ressourcenname-DNS-A-AAAA-Datensatz
Deaktiviert

Subnetz-ARN
arn:aws:ec2:us-east-1:767397790708:subnet/subnet-003febe3900aa2e0f

Verfügbare IPv4-Adressen
250

Network Border Group
us-east-1

Standard-Subnetz
Nein

Kundeneigener IPv4-Pool
-

Nur IPv6
Nein

DNS64
Deaktiviert

Status
Available

IPv6-CIDR
-

VPC
vpc-01642c1407e32f458 | tf-web-vpc

Öffentliche IPv4-Adresse automatisch zuweisen
Nein

Stützpunkt-ID
-

Hostname-Typ
IP-Name

Eigentümer
767397790708

Öffentlichen Zugriff blockieren
Aus

IPv6-CIDR-Zuordnungs-ID
-

Routing-Tabelle
rtb-09d58ee8e0b88609c | tf-web-rt

IPv6-Adresse automatisch zuweisen
Nein

IPv4 CIDR-Reservierungen
-

Ressourcenname-DNS-A-Datensatz
Deaktiviert

Flow-Protokolle | **Routing-Tabelle** | Netzwerk-ACL | CIDR-Reservierungen | Freigeben | Tags

Routing-Tabelle: rtb-09d58ee8e0b88609c / tf-web-rt Routing-Tabellenzuordnung bearbeiten

Routen (2)

Q Routen filtern

Bestimmungsort	Ziel
10.0.0.0/16	local
0.0.0.0/0	igw-0c40f497ee5149ed6

Abbildung 7: Subnet and Route Table Association

2.2 Compute and Access

2.2.1 Security Group

Similar to Workshop 1, we create a security group to allow HTTP traffic.

sg-06bad6ac684893f17 - tf-web-sg Aktionen

Details

Name der Sicherheitsgruppe
tf-web-sg

Besitzer
767397790708

Sicherheitsgruppen-ID
sg-06bad6ac684893f17

Anzahl der Regeln für eingehenden Datenverkehr
2 Berechtigungseingaben

Beschreibung
Allow HTTP in, all egress

Anzahl der Regeln für ausgehenden Datenverkehr
2 Berechtigungseingaben

VPC-ID
vpc-01642c1407e32f458

Regeln für eingehenden Datenverkehr | Regeln für ausgehenden Datenverkehr | Wird freigegeben ... - neu | VPC-Zuordnungen - neu | Tags

Regeln für eingehenden Datenverkehr (2) Tags verwalten Regeln für eingehenden Datenverkehr bearbeiten

Q Suchen

	Name	ID der Sicherheitsgr...	IP-Version	Typ	Protokoll	Portbereich
☐	-	sgr-082dfdda289601aa7	IPv4	HTTP	TCP	80
☐	-	sgr-0ceac4166dfa6c64f	IPv6	HTTP	TCP	80

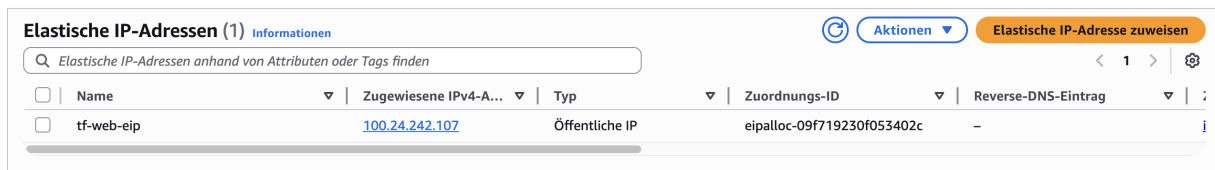
Abbildung 8: Security Group Inbound Rules

2.2.2 Network Interface and Elastic IP

Instead of relying on auto-assigned public IPs, we create a Network Interface (ENI) with a fixed private IP (10.0.1.50) and associate an Elastic IP (EIP) with it. This gives our server a static public IP address.

```
resource "aws_network_interface" "web_eni" {
  subnet_id      = aws_subnet.web_subnet.id
  private_ips    = ["10.0.1.50"]
  security_groups = [aws_security_group.web_sg.id]
  tags = { Name = "tf-web-eni" }
}

resource "aws_eip" "web_eip" {
  domain                = "vpc"
  network_interface     = aws_network_interface.web_eni.id
  associate_with_private_ip = "10.0.1.50"
  depends_on            = [aws_internet_gateway.igw]
  tags = { Name = "tf-web-eip" }
}
```



Name	Zugewiesene IPv4-Adresse	Typ	Zuordnungs-ID	Reverse-DNS-Eintrag
tf-web-eip	100.24.242.107	Öffentliche IP	eipalloc-09f719230f053402c	-

Abbildung 9: Elastic IP Allocation

2.2.3 EC2 Instance

We launch the EC2 instance and attach the previously created network interface. This ensures the instance uses our specific network configuration.

```
resource "aws_instance" "web" {
  ami          = data.aws_ami.ubuntu.id
  instance_type = "t2.micro"

  network_interface {
    network_interface_id = aws_network_interface.web_eni.id
    device_index         = 0
  }

  user_data = <<-EOF
    #!/bin/bash
    sudo apt-get update
```

```

sudo apt-get install -y apache2
sudo systemctl start apache2
sudo systemctl enable apache2
echo "<h1>Hello_World</h1>" | sudo tee /var/www/html/index.html
EOF

tags = { Name = "tf-apache-instance" }
}

```

Instance-Zusammenfassung für i-0b44d8e0decc8c1f3 (tf-apache-instance) [Info](#) [Verbinden](#) [Instance-Status](#) [Aktionen](#)

Vor less than a minute aktualisiert

Instance-ID i-0b44d8e0decc8c1f3	Öffentliche IPv4-Adresse 100.24.242.107 offene Adresse	Private IPv4-Adressen 10.0.1.50
IPv6-Adresse -	Instance-Status Läuft	Öffentlicher DNS ec2-100-24-242-107.compute-1.amazonaws.com offene Adresse
Hostname-Typ IP-Name: ip-10-0-1-50.ec2.internal	Private IP DNS-Name (nur IPv4) ip-10-0-1-50.ec2.internal	Elastic IP-Adressen 100.24.242.107 (tf-web-eip) [Öffentliche IP-Adresse]
DNS-Name der Antwort für private Ressourcen -	Instance-Typ t2.micro	AWS Compute Optimizer-Treffer Melden Sie sich für AWS Compute Optimizer an, um Empfehlungen zu erhalten. Weitere Informationen
Automatisch zugewiesene IP-Adresse -	VPC-ID vpc-01642c1407e32f458 (tf-web-vpc)	Auto-Scaling-Gruppenname -
IAM-Rolle -	Subnetz-ID subnet-003febe3900aa2e0f (tf-web-subnet)	Verwaltet Falsch
IMDSv2 Optional ⚠ EC2 empfiehlt, IMDSv2 auf erforderlichlich zu setzen. Weitere Informationen	Instance-ARN arn:aws:ec2:us-east-1:767397790708:instance/i-0b44d8e0decc8c1f3	
Operator -		

Abbildung 10: EC2 Instance Summary

2.3 Execution

We apply the configuration and verify the creation of resources. The output confirms the public IP of our instance.

```

> terraform fmt
> terraform validate
Success! The configuration is valid.

> terraform apply -auto-approve
data.aws_ami.ubuntu: Reading...
data.aws_ami.ubuntu: Read complete after 0s [id=ami-00b13f11600160c10]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following
symbols:
+ create

Terraform will perform the following actions:

# aws_eip.web_eip will be created
+ resource "aws_eip" "web_eip" {
+   allocation_id      = (known after apply)
+   arn                = (known after apply)
+   associate_with_private_ip = "10.0.1.50"
+   association_id     = (known after apply)
+   carrier_ip        = (known after apply)
+   customer_owned_ip  = (known after apply)
+   domain             = "vpc"
+   id                = (known after apply)
+   instance           = (known after apply)
+   ipam_pool_id       = (known after apply)
+   network_border_group = (known after apply)
+   network_interface  = (known after apply)
+   private_dns        = (known after apply)
+   private_ip         = (known after apply)
+   ptr_record         = (known after apply)
+   public_dns         = (known after apply)
+   public_ip          = (known after apply)
+   public_ipv4_pool    = (known after apply)
+   region             = "us-east-1"
+   tags               = {
+     "Name" = "tf-web-eip"
+   }
+   tags_all          = {
+     "Name" = "tf-web-eip"
+   }
}

```

Abbildung 11: Terraform Apply Execution